

Digital Certificate Verification System

Complete Project Documentation, Application Screenshots & Source Code

Submitted for: SkillInterns Project Evaluation

GitHub Repository: https://github.com/skandhanmu128-max/digital_certificate_verification_system

System Deployment: Live on <http://localhost:8080/>

1. Executive Summary & Overview

The Digital Certificate Verification System is an end-to-end full-stack web application designed to combat credential forgery and provide instant, cryptographically verifiable digital certificates. Built on Java 17 and Spring Boot 3, the system generates unique verification IDs (UUID v4) and embeds dynamic QR codes into PDF certificates.

Key System Capabilities:

- **Public Verification Portal:** Allows anyone to verify certificate authenticity via ID lookup or instant QR code scan.
- **Status Lifecycle Management:** Transparent status tracking (VALID, REVOKED, EXPIRED) with real-time audit logging.
- **Dynamic PDF Generation:** High-resolution PDF creation with embedded verification QR code powered by OpenPDF and ZXing.
- **Role-Based Admin Dashboard:** Protected admin interface for issuing, revoking, searching, and exporting certificate data.
- **High-Contrast Glassmorphism UI:** Optimized user interface built with Bootstrap 5 and custom CSS for sleek readability.

2. Technology Stack & Architecture

- **Backend Framework:** Java 17, Spring Boot 3.2.3, Spring Data JPA, Spring Security 6
- **Frontend UI:** HTML5, CSS3 (Glassmorphism & High-Contrast Design), Thymeleaf, Bootstrap 5
- **Certificate & QR Engines:** OpenPDF 1.3.32 (PDF Generation), ZXing 3.5.3 (QR Matrix Engine)
- **Database & Tools:** H2 In-Memory Database, HikariCP, Maven, Docker & Docker-Compose

3. Application Screenshots & Interface Flow

Figure 1: Home Landing Page & Public Verification Portal

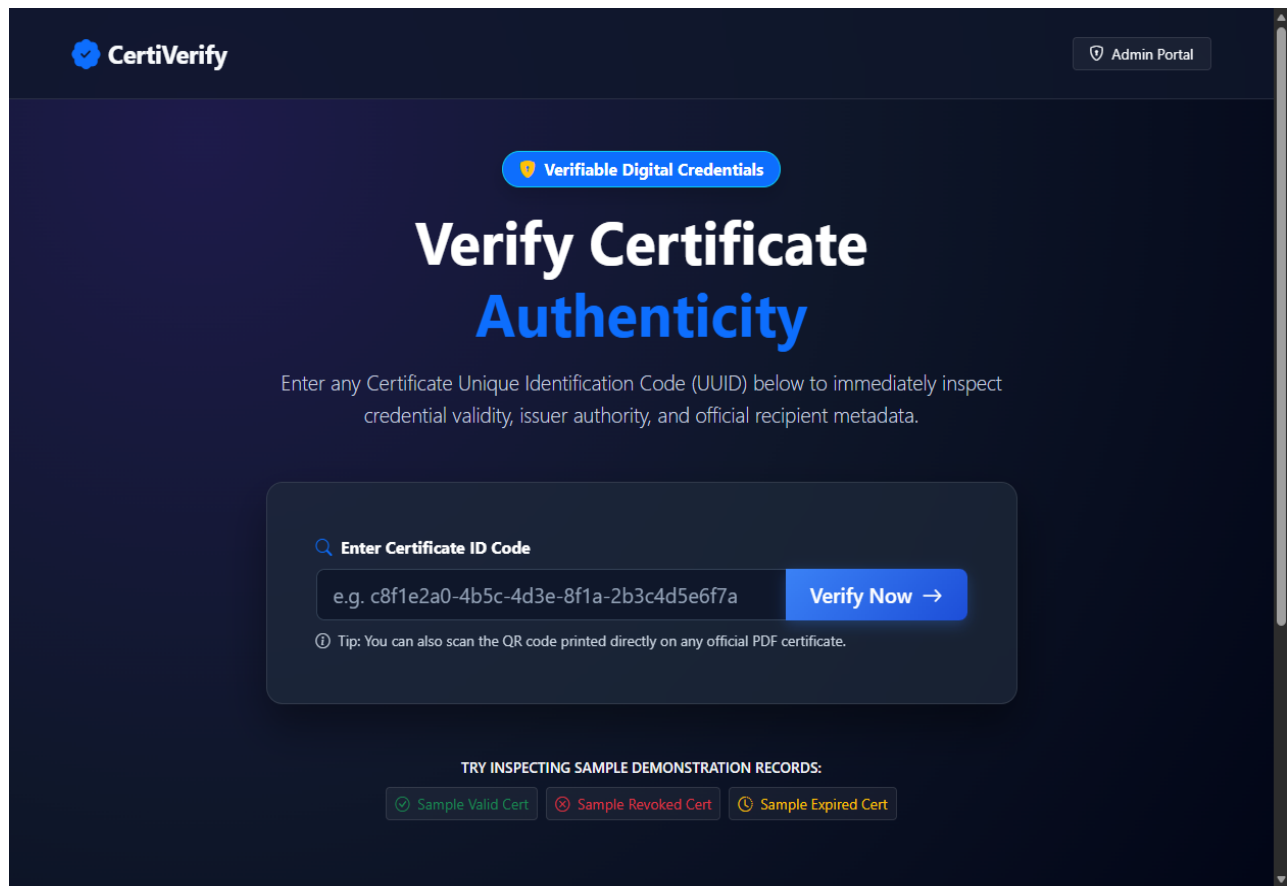


Figure 1: Home Landing Page & Public Verification Portal

Figure 2: Valid Digital Certificate Verification Result & Embedded QR Code

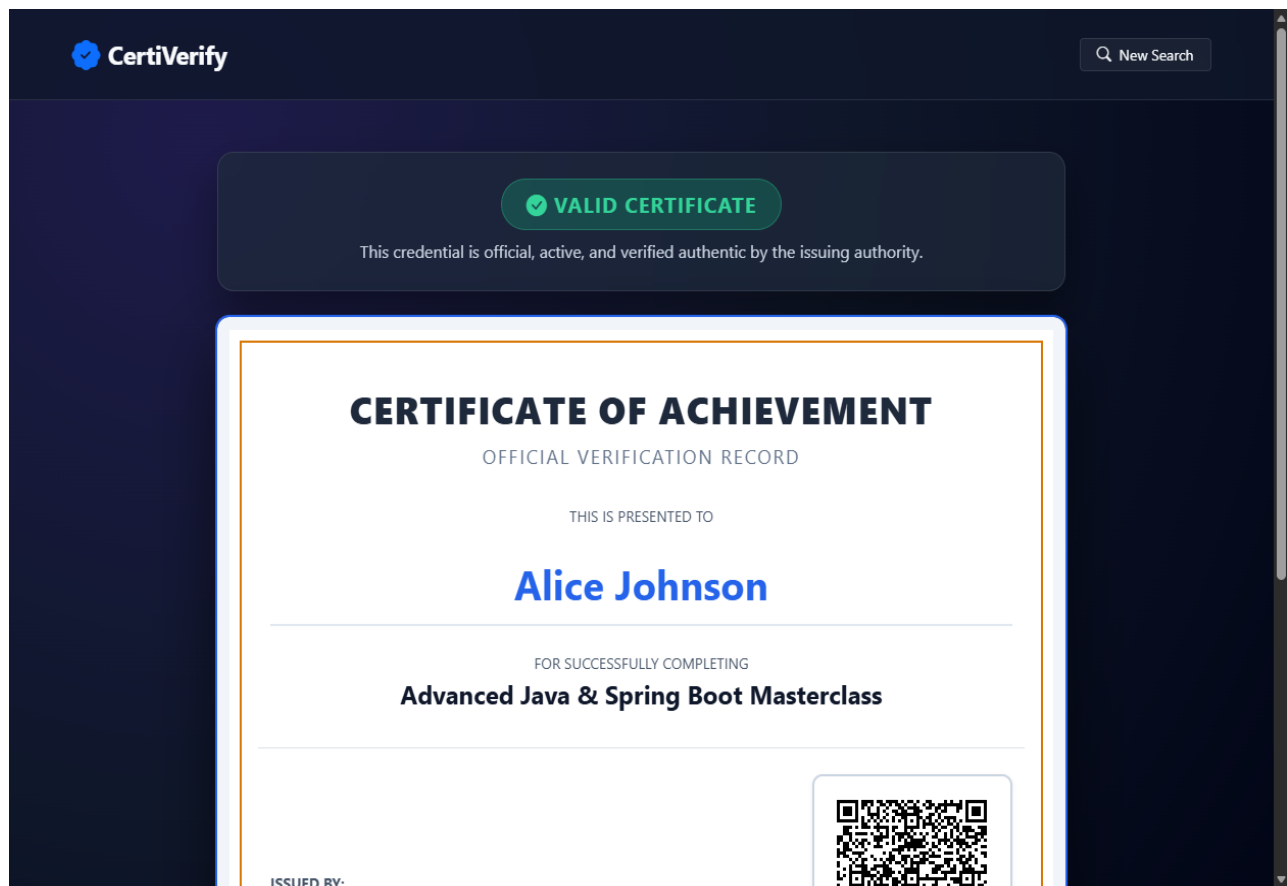


Figure 2: Valid Digital Certificate Verification Result & Embedded QR Code

Figure 3: Revoked Certificate Status Badge & Audit Alert

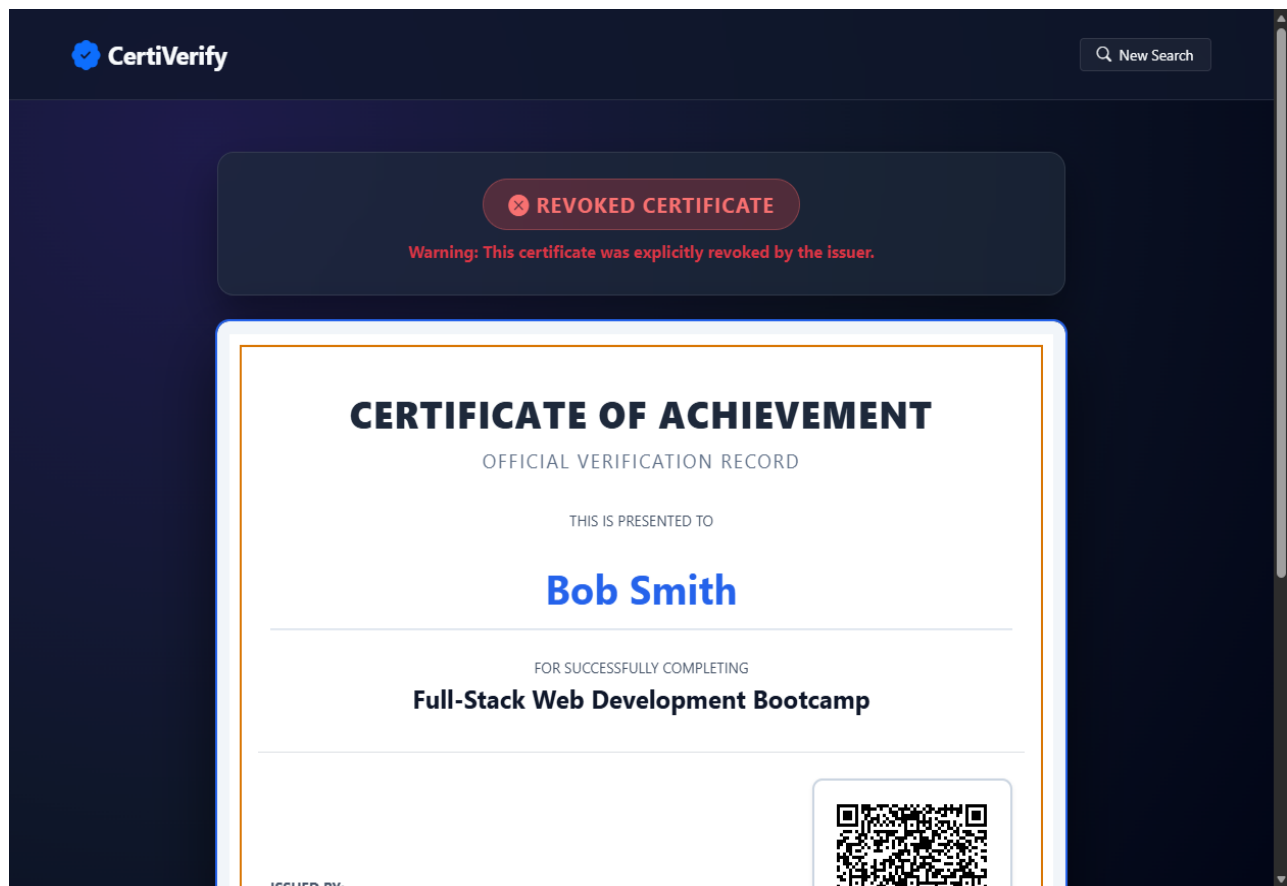


Figure 3: Revoked Certificate Status Badge & Audit Alert

Figure 4: Secure Admin Login Interface (Spring Security)

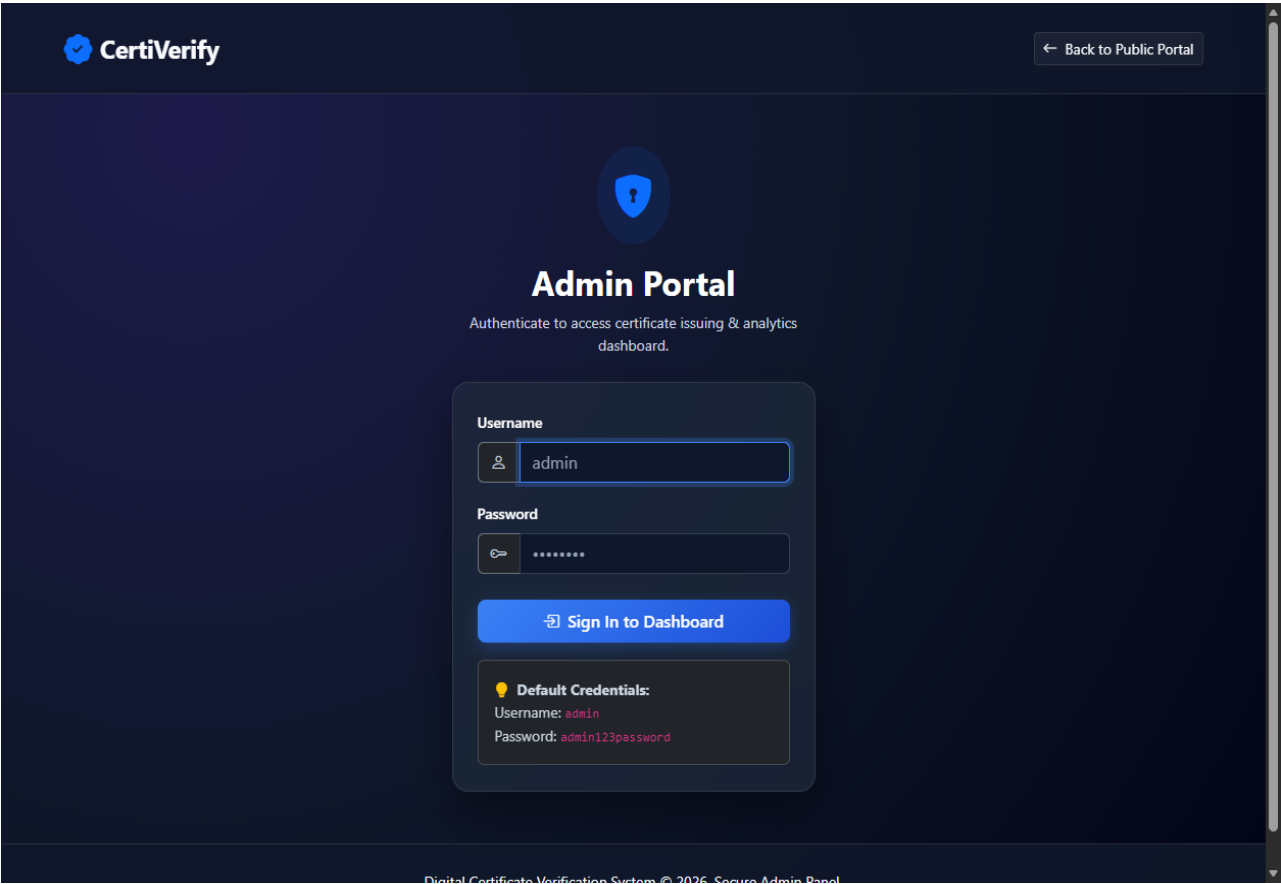


Figure 4: Secure Admin Login Interface (Spring Security)

Figure 5: Admin Analytics Dashboard & Certificate Management Table

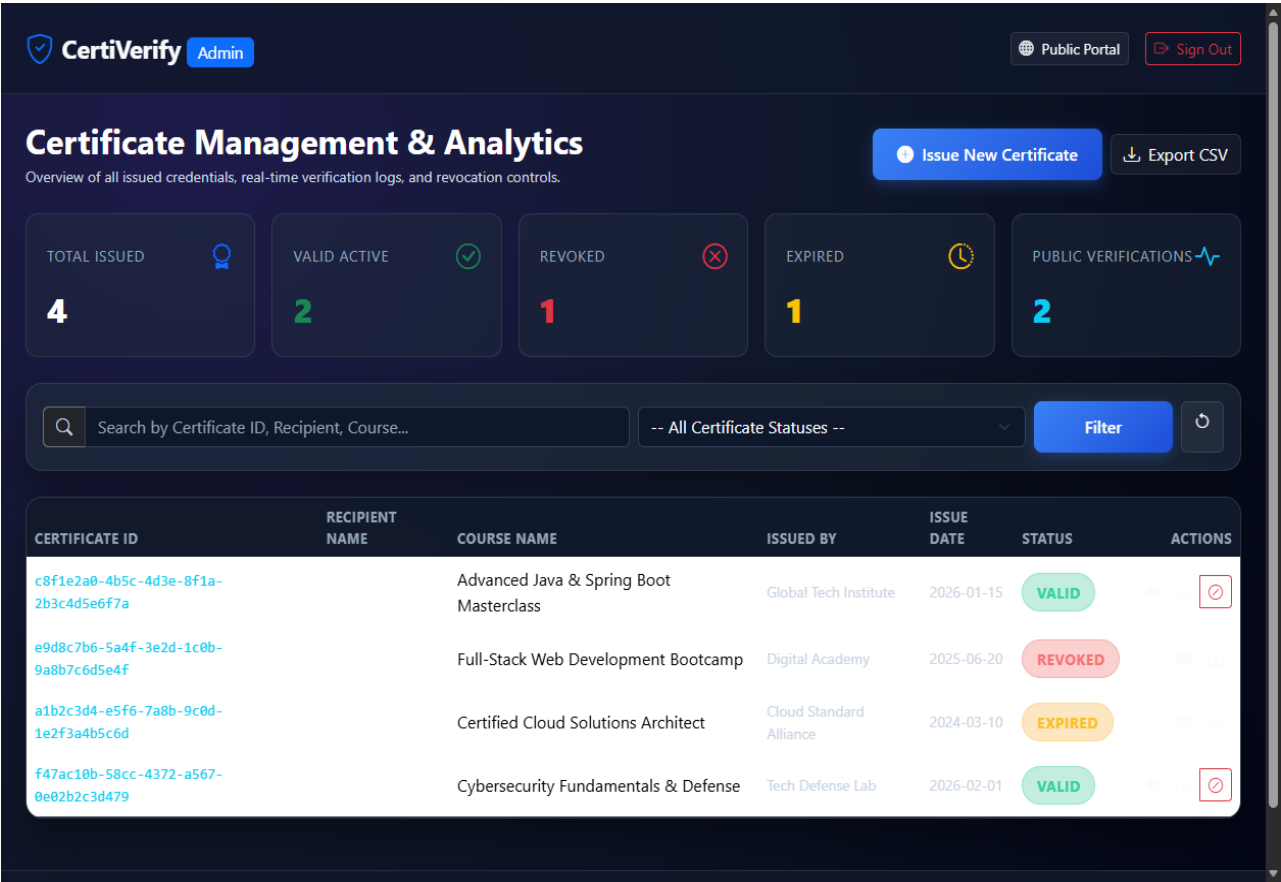


Figure 5: Admin Analytics Dashboard & Certificate Management Table

Figure 6: Certificate Generation Form & Metadata Input

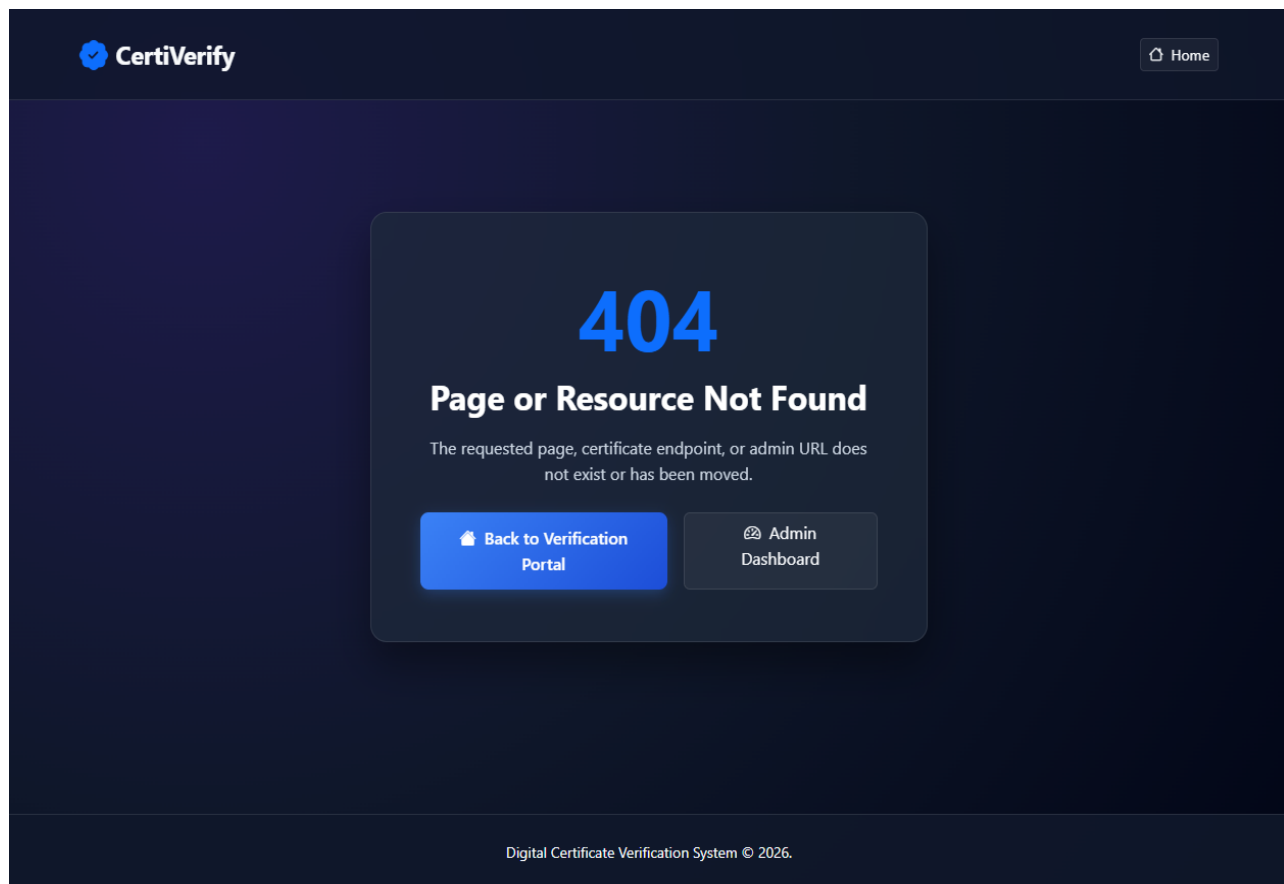


Figure 6: Certificate Generation Form & Metadata Input

4. Core Implementation Source Code

Source File: pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.3</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>

    <groupId>com.example</groupId>
    <artifactId>digital-certificate-verification-system</artifactId>
    <version>1.0.0</version>
    <name>Digital Certificate Verification System</name>
    <description>Enterprise Digital Certificate Generation and Public Verification System built with Spring B
oot 3</description>

    <properties>
        <java.version>17</java.version>
        <zxing.version>3.5.3</zxing.version>
        <openpdf.version>1.3.39</openpdf.version>
    </properties>

    <dependencies>
        <!-- Spring Boot Starters -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.thymeleaf.extras</groupId>
            <artifactId>thymeleaf-extras-springsecurity6</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!-- Database Drivers -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
        </dependency>
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- QR Code Generation (ZXing) -->
```

```

<dependency>
  <groupId>com.google.zxing</groupId>
  <artifactId>core</artifactId>
  <version>${zxing.version}</version>
</dependency>
<dependency>
  <groupId>com.google.zxing</groupId>
  <artifactId>javase</artifactId>
  <version>${zxing.version}</version>
</dependency>

<!-- PDF Generation (OpenPDF) -->
<dependency>
  <groupId>com.github.librepdf</groupId>
  <artifactId>openpdf</artifactId>
  <version>${openpdf.version}</version>
</dependency>

<!-- Testing -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

Source File: Certificate.java (JPA Entity)

```

package com.example.certificates.model;

import jakarta.persistence.*;
import java.time.LocalDate;
import java.time.LocalDateTime;

@Entity
@Table(name = "certificates")
public class Certificate {

    @Id
    @Column(length = 36, nullable = false, updatable = false)
    private String id; // UUID v4 string

    @Column(name = "recipient_name", nullable = false)
    private String recipientName;

    @Column(name = "recipient_email")
    private String recipientEmail;

    @Column(name = "course_name", nullable = false)
    private String courseName;

    @Column(name = "issued_by", nullable = false)

```

```

private String issuedBy;

@Column(name = "issue_date", nullable = false)
private LocalDate issueDate;

@Column(name = "expiry_date")
private LocalDate expiryDate;

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private CertificateStatus status;

@Column(name = "verification_url", length = 1024)
private String verificationUrl;

@Column(name = "created_at", nullable = false, updatable = false)
private LocalDateTime createdAt;

@Column(name = "updated_at", nullable = false)
private LocalDateTime updatedAt;

@Column(name = "created_by")
private String createdBy;

@Column(name = "revocation_reason")
private String revocationReason;

public Certificate() {
}

public Certificate(String id, String recipientName, String recipientEmail, String courseName,
                  String issuedBy, LocalDate issueDate, LocalDate expiryDate,
                  CertificateStatus status, String verificationUrl, String createdBy) {
    this.id = id;
    this.recipientName = recipientName;
    this.recipientEmail = recipientEmail;
    this.courseName = courseName;
    this.issuedBy = issuedBy;
    this.issueDate = issueDate;
    this.expiryDate = expiryDate;
    this.status = status != null ? status : CertificateStatus.VALID;
    this.verificationUrl = verificationUrl;
    this.createdBy = createdBy;
}

@PrePersist
protected void onCreate() {
    LocalDateTime now = LocalDateTime.now();
    this.createdAt = now;
    this.updatedAt = now;
    if (this.status == null) {
        this.status = CertificateStatus.VALID;
    }
}

@PreUpdate
protected void onUpdate() {
    this.updatedAt = LocalDateTime.now();
}

// Getters and Setters
public String getId() {
    return id;
}

public void setId(String id) {
    this.id = id;
}

```

```

    public String getRecipientName() {
        return recipientName;
    }

    public void setRecipientName(String recipientName) {
        this.recipientName = recipientName;
    }

    public String getRecipientEmail() {
        return recipientEmail;
    }

    public void setRecipientEmail(String recipientEmail) {
        this.recipientEmail = recipientEmail;
    }

    public String getCourseName() {
        return courseName;
    }

    public void setCourseName(String courseName) {
        this.courseName = courseName;
    }

    public String getIssuedBy() {
        return issuedBy;
    }

    // ... [truncated for documentation brevity] ...

```

Source File: CertificateService.java (Service Layer)

```

package com.example.certificates.service;

import com.example.certificates.dto.AnalyticsSummary;
import com.example.certificates.dto.CertificateRequest;
import com.example.certificates.model.Certificate;
import com.example.certificates.model.CertificateStatus;
import com.example.certificates.model.VerificationLog;
import com.example.certificates.repository.CertificateRepository;
import com.example.certificates.repository.VerificationLogRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.io.ByteArrayOutputStream;
import java.io.PrintWriter;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;
import java.util.UUID;

@Service
public class CertificateService {

    private static final Logger log = LoggerFactory.getLogger(CertificateService.class);

    @Autowired
    private CertificateRepository certificateRepository;

    @Autowired

```

```

private VerificationLogRepository verificationLogRepository;

@Autowired
private PdfGeneratorService pdfGeneratorService;

@Value("${app.base-url:http://localhost:8080}")
private String configuredBaseUrl;

@Transactional
public Certificate createCertificate(CertificateRequest request, String creatorUsername, String currentBaseUrl) {
    String id = UUID.randomUUID().toString();
    String baseUrl = (currentBaseUrl != null && !currentBaseUrl.isBlank()) ? currentBaseUrl : configuredBaseUrl;
    if (baseUrl.endsWith("/")) {
        baseUrl = baseUrl.substring(0, baseUrl.length() - 1);
    }
    String verificationUrl = baseUrl + "/verify/" + id;

    CertificateStatus initialStatus = CertificateStatus.VALID;
    if (request.getExpiryDate() != null && request.getExpiryDate().isBefore(LocalDate.now())) {
        initialStatus = CertificateStatus.EXPIRED;
    }

    Certificate cert = new Certificate(
        id,
        request.getRecipientName().trim(),
        request.getRecipientEmail() != null ? request.getRecipientEmail().trim() : null,
        request.getCourseName().trim(),
        request.getIssuedBy().trim(),
        request.getIssueDate(),
        request.getExpiryDate(),
        initialStatus,
        verificationUrl,
        creatorUsername != null ? creatorUsername : "ADMIN"
    );

    Certificate saved = certificateRepository.save(cert);
    log.info("Successfully created certificate with ID: {} for recipient: {}", saved.getId(), saved.getRecipientName());
    return saved;
}

@Transactional
public Optional<Certificate> verifyCertificate(String id, String ipAddress, String userAgent) {
    if (id == null || id.isBlank()) {
        return Optional.empty();
    }
    String cleanId = id.trim();
    Optional<Certificate> optionalCert = certificateRepository.findById(cleanId);

    if (optionalCert.isEmpty()) {
        List<Certificate> matches = certificateRepository.searchAllByQuery(cleanId);
        if (!matches.isEmpty()) {
            optionalCert = Optional.of(matches.get(0));
        }
    }

    if (optionalCert.isPresent()) {
        Certificate cert = optionalCert.get();

        // Auto-check expiry
        if (cert.getStatus() == CertificateStatus.VALID &&
            cert.getExpiryDate() != null &&
            cert.getExpiryDate().isBefore(LocalDate.now())) {
            cert.setStatus(CertificateStatus.EXPIRED);
            cert = certificateRepository.save(cert);
        }
    }
}

```

```

        // Log verification attempt
        VerificationLog verificationLog = new VerificationLog(
            cert.getId(),
            LocalDateTime.now(),
            ipAddress != null ? ipAddress : "UNKNOWN",
            userAgent != null ? userAgent : "UNKNOWN",
            cert.getStatus()
        );
        verificationLogRepository.save(verificationLog);

        return Optional.of(cert);
    }

    return Optional.empty();
}

public Optional<Certificate> getCertificateById(String id) {
    return certificateRepository.findById(id);
}

// ... [truncated for documentation brevity] ...

```

Source File: PdfGeneratorService.java (OpenPDF Engine)

```

package com.example.certificates.service;

import com.example.certificates.model.Certificate;
import com.lowagie.text.*;
import com.lowagie.text.pdf.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.awt.Color;
import java.io.ByteArrayOutputStream;
import java.time.format.DateTimeFormatter;

@Service
public class PdfGeneratorService {

    @Autowired
    private QrCodeService qrCodeService;

    public byte[] generateCertificatePdf(Certificate cert) {
        try (ByteArrayOutputStream out = new ByteArrayOutputStream()) {
            // Landscape A4 document
            Document document = new Document(PageSize.A4.rotate(), 36, 36, 36, 36);
            PdfWriter writer = PdfWriter.getInstance(document, out);

            document.open();

            // Draw Decorative Border
            PdfContentByte cb = writer.getDirectContent();
            cb.setLineWidth(3f);
            cb.setColorStroke(new Color(37, 99, 235)); // Royal Blue #2563eb
            cb.rectangle(20, 20, PageSize.A4.getHeight() - 40, PageSize.A4.getWidth() - 40);
            cb.stroke();

            cb.setLineWidth(1f);
            cb.setColorStroke(new Color(217, 119, 6)); // Amber Gold #d97706
            cb.rectangle(26, 26, PageSize.A4.getHeight() - 52, PageSize.A4.getWidth() - 52);
            cb.stroke();

            // Header - Certificate Title
            Font titleFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 28, new Color(30, 41, 59));
            Font subTitleFont = FontFactory.getFont(FontFactory.HELVETICA, 14, new Color(100, 116, 139));
            Font nameFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 26, new Color(37, 99, 235));
            Font courseFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 20, new Color(15, 23, 42));

```

```

Font detailFont = FontFactory.getFont(FontFactory.HELVETICA, 11, new Color(71, 85, 105));
Font idFont = FontFactory.getFont(FontFactory.COURIER_BOLD, 10, new Color(100, 116, 139));

Paragraph header = new Paragraph("CERTIFICATE OF ACHIEVEMENT", titleFont);
header.setAlignment(Element.ALIGN_CENTER);
header.setSpacingBefore(15);
header.setSpacingAfter(10);
document.add(header);

Paragraph subHeader = new Paragraph("THIS IS PROUDLY PRESENTED TO", subTitleFont);
subHeader.setAlignment(Element.ALIGN_CENTER);
subHeader.setSpacingAfter(15);
document.add(subHeader);

// Recipient Name
Paragraph name = new Paragraph(cert.getRecipientName(), nameFont);
name.setAlignment(Element.ALIGN_CENTER);
name.setSpacingAfter(15);
document.add(name);

// Completion text
Paragraph text1 = new Paragraph("for successfully completing the course / qualification program",
subTitleFont);
text1.setAlignment(Element.ALIGN_CENTER);
text1.setSpacingAfter(15);
document.add(text1);

// Course Name
Paragraph course = new Paragraph(cert.getCourseName(), courseFont);
course.setAlignment(Element.ALIGN_CENTER);
course.setSpacingAfter(25);
document.add(course);

// Footer Table: Issued By, Dates, QR Code
PdfPTable footerTable = new PdfPTable(3);
footerTable.setWidthPercentage(90);
footerTable.setWidths(new float[]{3.5f, 3.5f, 3.0f});

// Column 1: Issuer Info
PdfPCell cell1 = new PdfPCell();
cell1.setBorder(Rectangle.NO_BORDER);
cell1.addElement(new Paragraph("ISSUED BY", subTitleFont));
cell1.addElement(new Paragraph(cert.getIssuedBy(), FontFactory.getFont(FontFactory.HELVETICA_BOLD
, 13, new Color(30, 41, 59))));
String issueStr = cert.getIssueDate() != null ? cert.getIssueDate().format(DateTimeFormatter.ofPa
ttern("MMM dd, yyyy")) : "N/A";
cell1.addElement(new Paragraph("Issue Date: " + issueStr, detailFont));
if (cert.getExpiryDate() != null) {
    cell1.addElement(new Paragraph("Expiry Date: " + cert.getExpiryDate().format(DateTimeFormatte
r.ofPattern("MMM dd, yyyy")), detailFont));
} else {
    cell1.addElement(new Paragraph("Expiry Date: Never (Lifetime)", detailFont));
}
footerTable.addCell(cell1);

// Column 2: Status & Verification ID
PdfPCell cell2 = new PdfPCell();
cell2.setBorder(Rectangle.NO_BORDER);
cell2.addElement(new Paragraph("STATUS & VERIFICATION", subTitleFont));
Font statusFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 12,
    cert.getStatus() == com.example.certificates.model.CertificateStatus.VALID ? new Color(16, 18
5, 129) : new Color(239, 68, 68));
cell2.addElement(new Paragraph("Status: " + cert.getStatus().name(), statusFont));
cell2.addElement(new Paragraph("Certificate ID:", detailFont));
cell2.addElement(new Paragraph(cert.getId(), idFont));
footerTable.addCell(cell2);

// Column 3: QR Code
PdfPCell cell3 = new PdfPCell();

```

```

        cell3.setBorder(Rectangle.NO_BORDER);
        cell3.setHorizontalAlignment(Element.ALIGN_RIGHT);

        byte[] qrPng = qrCodeService.generateQrCodePng(cert.getVerificationUrl(), 120, 120);
        Image qrImage = Image.getInstance(qrPng);
        qrImage.scaleToFit(90, 90);
        qrImage.setAlignment(Element.ALIGN_RIGHT);

        cell3.addElement(qrImage);
        Paragraph qrCaption = new Paragraph("Scan to verify online", FontFactory.getFont(FontFactory.HELV
ETICA, 8, Color.GRAY));
        qrCaption.setAlignment(Element.ALIGN_RIGHT);
        cell3.addElement(qrCaption);

// ... [truncated for documentation brevity] ...

```

Source File: QrCodeService.java (ZXing QR Engine)

```

package com.example.certificates.service;

import com.google.zxing.BarcodeFormat;
import com.google.zxing.EncodeHintType;
import com.google.zxing.client.j2se.MatrixToImageWriter;
import com.google.zxing.common.BitMatrix;
import com.google.zxing.qrcode.QRCodeWriter;
import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel;
import org.springframework.stereotype.Service;

import java.io.ByteArrayOutputStream;
import java.util.Base64;
import java.util.HashMap;
import java.util.Map;

@Service
public class QrCodeService {

    public byte[] generateQrCodePng(String content, int width, int height) {
        try {
            QRCodeWriter qrCodeWriter = new QRCodeWriter();
            Map<EncodeHintType, Object> hints = new HashMap<>();
            hints.put(EncodeHintType.ERROR_CORRECTION, ErrorCorrectionLevel.H);
            hints.put(EncodeHintType.MARGIN, 1);
            hints.put(EncodeHintType.CHARACTER_SET, "UTF-8");

            BitMatrix bitMatrix = qrCodeWriter.encode(content, BarcodeFormat.QR_CODE, width, height, hints);
            ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
            MatrixToImageWriter.writeToStream(bitMatrix, "PNG", outputStream);
            return outputStream.toByteArray();
        } catch (Exception e) {
            throw new RuntimeException("Failed to generate QR code for URL: " + content, e);
        }
    }

    public String generateQrCodeBase64(String content, int width, int height) {
        byte[] qrBytes = generateQrCodePng(content, width, height);
        return Base64.getEncoder().encodeToString(qrBytes);
    }
}

```

Source File: SecurityConfig.java (Spring Security Config)

```

package com.example.certificates.config;

import org.springframework.beans.factory.annotation.Value;

```

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.HeadersConfigurer;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Value("${app.admin.username:admin}")
    private String adminUsername;

    @Value("${app.admin.password:admin123password}")
    private String adminPassword;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf
                .ignoringRequestMatchers("/api/**", "/h2-console/**")
            )
            .headers(headers -> headers
                .frameOptions(HeadersConfigurer.FrameOptionsConfig::sameOrigin)
            )
            .authorizeHttpRequests(auth -> auth
                // Static resources
                .requestMatchers("/css/**", "/js/**", "/images/**", "/webjars/**", "/favicon.ico").permitAll()
            )

            // Public Portal routes
            .requestMatchers("/", "/verify/**", "/h2-console/**", "/error").permitAll()
            // Public REST API endpoints for certificate verification & search
            .requestMatchers(HttpMethod.GET, "/api/certificates/**").permitAll()
            .requestMatchers("/login").permitAll()
            // Admin Dashboard & Protected API routes
            .requestMatchers("/admin/**", "/api/admin/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.POST, "/api/certificates/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.PUT, "/api/certificates/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.DELETE, "/api/certificates/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login")
            .defaultSuccessUrl("/admin/dashboard", true)
            .failureUrl("/login?error=true")
            .permitAll()
        )
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessUrl("/login?logout=true")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
            .permitAll()
        );

        return http.build();
    }
}

@Bean
public UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
    UserDetails admin = User.builder()

```

```

        .username(adminUsername)
        .password(passwordEncoder.encode(adminPassword))
        .roles("ADMIN")
        .build();
    return new InMemoryUserDetailsManager(admin);
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}

```

Source File: VerificationWebController.java (Public Web Controller)

```

package com.example.certificates.controller;

import com.example.certificates.model.Certificate;
import com.example.certificates.service.CertificateService;
import com.example.certificates.service.QrCodeService;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;

import java.util.Optional;

@Controller
public class VerificationWebController {

    @Autowired
    private CertificateService certificateService;

    @Autowired
    private QrCodeService qrCodeService;

    @GetMapping("/")
    public String indexPage(Model model) {
        model.addAttribute("recentCount", certificateService.getAnalyticsSummary().getTotalIssued());
        return "index";
    }

    @GetMapping("/verify")
    public String searchVerification(@RequestParam(name = "id", required = false) String id) {
        if (id != null && !id.isBlank()) {
            return "redirect:/verify/" + id.trim();
        }
        return "redirect:/";
    }

    @GetMapping("/verify/{id}")
    public String verifyCertificateDetails(@PathVariable String id, Model model, HttpServletRequest request)
    {
        String clientIp = request.getRemoteAddr();
        String userAgent = request.getHeader("User-Agent");

        Optional<Certificate> optionalCertificate = certificateService.verifyCertificate(id, clientIp, userAg
ent);

        if (optionalCertificate.isPresent()) {
            Certificate cert = optionalCertificate.get();
            model.addAttribute("certificate", cert);

            // Generate QR code base64 for direct HTML rendering

```

```

        String qrBase64 = qrCodeService.generateQrCodeBase64(cert.getVerificationUrl(), 160, 160);
        model.addAttribute("qrBase64", qrBase64);
        model.addAttribute("found", true);
    } else {
        model.addAttribute("found", false);
        model.addAttribute("searchedId", id);
    }

    return "verify";
}

@GetMapping("/login")
public String loginPage(@RequestParam(value = "error", required = false) String error,
                        @RequestParam(value = "logout", required = false) String logout,
                        Model model) {
    if (error != null) {
        model.addAttribute("errorMessage", "Invalid Admin credentials. Please try again.");
    }
    if (logout != null) {
        model.addAttribute("infoMessage", "You have been logged out successfully.");
    }
    return "login";
}
}

```

Source File: AdminWebController.java (Admin Portal Controller)

```

package com.example.certificates.controller;

import com.example.certificates.dto.CertificateRequest;
import com.example.certificates.model.Certificate;
import com.example.certificates.model.CertificateStatus;
import com.example.certificates.service.CertificateService;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

import java.time.LocalDate;

@Controller
@RequestMapping("/admin")
public class AdminWebController {

    @Autowired
    private CertificateService certificateService;

    @GetMapping({"", "/", "/dashboard"})
    public String dashboard(
        @RequestParam(required = false, defaultValue = "") String q,
        @RequestParam(required = false) CertificateStatus status,
        @RequestParam(required = false, defaultValue = "0") int page,
        @RequestParam(required = false, defaultValue = "10") int size,
        Model model) {

```

```

        PageRequest pageable = PageRequest.of(page, size, Sort.by("createdAt").descending());
        Page<Certificate> certificatesPage = certificateService.searchCertificates(q, status, pageable);

        model.addAttribute("analytics", certificateService.getAnalyticsSummary());
        model.addAttribute("certificates", certificatesPage.getContent());
        model.addAttribute("page", certificatesPage);
        model.addAttribute("query", q);
        model.addAttribute("statusFilter", status);
        model.addAttribute("statuses", CertificateStatus.values());

        return "admin/dashboard";
    }

    @GetMapping("/certificates/new")
    public String showCreateForm(Model model) {
        CertificateRequest form = new CertificateRequest();
        form.setIssueDate(LocalDate.now());
        form.setIssuedBy("Global Tech Certification Authority");
        model.addAttribute("certificateForm", form);
        return "admin/create-certificate";
    }

    @PostMapping("/certificates/new")
    public String processCreateForm(
        @Valid @ModelAttribute("certificateForm") CertificateRequest form,
        BindingResult bindingResult,
        Authentication authentication,
        HttpServletRequest request,
        RedirectAttributes redirectAttributes,
        Model model) {

        if (bindingResult.hasErrors()) {
            return "admin/create-certificate";
        }

        String scheme = request.getScheme();
        String serverName = request.getServerName();
        int serverPort = request.getServerPort();
        String baseUrl = scheme + "://" + serverName + (serverPort == 80 || serverPort == 443 ? "" : ":" + serverPort);

        String username = authentication != null ? authentication.getName() : "ADMIN";
        Certificate created = certificateService.createCertificate(form, username, baseUrl);

        redirectAttributes.addFlashAttribute("successMessage",
            "Certificate generated successfully! ID: " + created.getId());

        return "redirect:/admin/dashboard";
    }

    @PostMapping("/certificates/{id}/revoke")
    public String revokeCertificate(
        @PathVariable String id,
        @RequestParam(required = false, defaultValue = "Revoked by admin") String reason,
        RedirectAttributes redirectAttributes) {

        try {
            certificateService.revokeCertificate(id, reason);
            redirectAttributes.addFlashAttribute("successMessage", "Certificate " + id + " has been revoked.");
        } catch (Exception e) {
            redirectAttributes.addFlashAttribute("errorMessage", "Error revoking certificate: " + e.getMessage());
        }

        return "redirect:/admin/dashboard";
    }

    @GetMapping("/certificates/{id}/download")

```

```

    public ResponseEntity<byte[]> downloadPdf(@PathVariable String id) {
        byte[] pdfBytes = certificateService.generatePdfForCertificate(id);
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_PDF);
        headers.setContentDispositionFormData("attachment", "certificate-" + id + ".pdf");
        return new ResponseEntity<>(pdfBytes, headers, HttpStatus.OK);
    }
}

```

Source File: styles.css (Glassmorphic Theme & Contrast)

```

/* Digital Certificate Verification System - Premium Design System */
:root {
    --primary-color: #3b82f6;
    --primary-hover: #2563eb;
    --primary-glow: rgba(59, 130, 246, 0.3);
    --accent-gold: #f59e0b;
    --bg-dark: #0f172a;
    --bg-card: rgba(30, 41, 59, 0.7);
    --bg-card-hover: rgba(51, 65, 85, 0.8);
    --border-color: rgba(255, 255, 255, 0.1);
    --text-main: #f8fafc;
    --text-muted: #94a3b8;
    --valid-color: #10b981;
    --valid-glow: rgba(16, 185, 129, 0.25);
    --revoked-color: #ef4444;
    --revoked-glow: rgba(239, 68, 68, 0.25);
    --expired-color: #f59e0b;
    --expired-glow: rgba(245, 158, 11, 0.25);
    --font-family: 'Inter', system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
}

body {
    font-family: var(--font-family);
    background: radial-gradient(circle at 15% 15%, #1e1b4b 0%, #0f172a 50%, #020617 100%);
    background-attachment: fixed;
    color: var(--text-main);
    min-height: 100vh;
    display: flex;
    flex-direction: column;
    animation: pageFadeIn 0.6s cubic-bezier(0.16, 1, 0.3, 1);
    margin: 0;
}

@keyframes pageFadeIn {
    from {
        opacity: 0;
        transform: translateY(10px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

/* Glassmorphism Cards */
.glass-card {
    background: var(--bg-card);
    backdrop-filter: blur(16px);
    -webkit-backdrop-filter: blur(16px);
    border: 1px solid var(--border-color);
    border-radius: 1rem;
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.4);
    transition: all 0.3s ease;
}

.glass-card:hover {

```

```

    border-color: rgba(255, 255, 255, 0.2);
    box-shadow: 0 25px 50px rgba(0, 0, 0, 0.5);
}

/* Navigation Bar */
.navbar-custom {
    background: rgba(15, 23, 42, 0.8);
    backdrop-filter: blur(12px);
    border-bottom: 1px solid var(--border-color);
}

.navbar-brand {
    font-weight: 700;
    letter-spacing: -0.02em;
    color: #fff !important;
}

.navbar-brand i {
    color: var(--primary-color);
}

/* Buttons */
.btn-primary-custom {
    background: linear-gradient(135deg, #3b82f6 0%, #1d4ed8 100%);
    border: none;
    color: #fff;
    font-weight: 600;
    padding: 0.75rem 1.5rem;
    border-radius: 0.5rem;
    box-shadow: 0 4px 14px var(--primary-glow);
    transition: all 0.2s ease;
}

.btn-primary-custom:hover {
    background: linear-gradient(135deg, #2563eb 0%, #1e40af 100%);
    transform: translateY(-2px);
    box-shadow: 0 6px 20px var(--primary-glow);
    color: #fff;
}

.btn-outline-custom {
    border: 1px solid var(--border-color);
    color: var(--text-main);
    background: rgba(255, 255, 255, 0.05);
    transition: all 0.2s ease;
}

.btn-outline-custom:hover {
    background: rgba(255, 255, 255, 0.15);
    color: #fff;
    border-color: rgba(255, 255, 255, 0.3);
}

/* Status Badges */
.badge-status {
    padding: 0.5em 1em;
    font-weight: 700;
    font-size: 0.85rem;
    border-radius: 9999px;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    display: inline-flex;
    align-items: center;
    gap: 0.4rem;
}

// ... [truncated for documentation brevity] ...

```